Cost-Aware Reinforcement Learning for Agentic RAG

Parsuram Pradhan

Abstract

Agentic RAG controllers decide when to rewrite, rerank, verify or search again, and each of those tools costs money. We ask whether a small learned controller, rewarded in dollars, spends a tool only when the tool can change the answer. With a frozen 3B generator and BM25 over 83,120 HotpotQA and Natural Questions passages, we find three things. First, the flat per-action charge decides tool use, not the dollar price: using every tool adds 5 correct answers out of 300 and still lowers the reward. Second, the logged trajectories leave a 26-answer gap above one-shot retrieval, and neither fixed recipes nor question-only routing recovers it. Third, REINFORCE controllers that see retrieval scores before they act also miss that gap. About a third of their bill goes to actions that cannot change the answer, because the dollar term is too small to steer the policy until it is large enough to collapse it to one-shot retrieval. Masking those actions lowers that share to about a quarter and does not close the gap. Our results point to the reward as the bottleneck, not the controller.

1 Introduction

On the Natural Questions item "what is the name of wonder womans mother," one-shot retrieval returns passages titled Seoul Korea Temple and Flash in other media, and the generator answers Stevie Wonder. Two rewrites, together 2.4×10^{-5} dollars, change the query to "mother of wonder woman." The next search returns Wonder Girl and Duke of Deception, and the answer is Queen Hippolyta. The episode costs 2.06×10^{-4} dollars, against 1.83×10^{-4} for one-shot retrieval. On this question the rewrite pays. On others it does not.

That is one question. Figure 1 is the whole pipeline. The generator and the retriever are frozen. Every number below is on one index of 83,120 passages.

The flat action charge decides tool use: it rises by more than the quality terms, so the reward falls as more tools are used. The gap among the trajectories we logged is 26 answers, from 100/300 to an oracle of 126/300, and a fixed recipe or question-only routing does not capture it. Learned controllers, including retrieve-first, miss that gap, and the reward leaves them nearly blind to cost until λ=80: they pay for actions that cannot change the answer, and no weight spends a tool only when it can.

Copyright © 2027, Association for the Advancement of Artificial Intelligence (www.aaai.org). All rights reserved.

2 Related Work

Retrieval-augmented generation conditions a generator on retrieved text (Lewis et al. 2020). Agentic controllers interleave that retrieval with other tools. ReAct alternates reasoning traces and actions (Yao et al. 2023). The five systems in Table 1 are the ones a reader will set this paper against. Each decides something about retrieval. None prices rewrite, rerank, and verify in dollars inside a closed loop.

Search-R1 trains the language model with an outcome reward to issue searches and reason over the results (Jin et al. 2025). GRASP is the closest typed-action neighbor: a GRPO policy over semantic search, keyword search, and paragraph reading, rewarded for answer F1, grounded reading, complementary search, and turn efficiency (Gandhi et al. 2026). The efficiency term is paid only when answer F1 exceeds 0.5, and it counts turns, not dollars. GRASP does not price rewrite, rerank, or verification. We keep the generator fixed and ask a prior question: once those tools have a dollar cost, does a learned controller spend them only when they pay?

FrugalGPT and RouteLLM price a different choice. FrugalGPT cascades queries across LLM APIs whose fees differ by orders of magnitude, so a cheap model answers when it can and an expensive model answers when it must (Chen, Zaharia, and Zou 2023). RouteLLM learns a router from preference data and sends each query to a strong model or a weak one (Ong et al. 2024). Both decide which model to call. We use one generator, Qwen2.5-3B-Instruct, and price the tool calls inside the episode.

The retrieval line behind that generator is older than the agent. REALM trains a retriever inside language-model pretraining (Guu et al. 2020). Fusion-in-Decoder reads each retrieved passage on its own and fuses them in the decoder (Izacard and Grave 2020). RETRO conditions each chunk of the generation on neighbors from a trillion-token store (Borgeaud et al. 2021). Atlas asks how far a retrieval-augmented model can go in the few-shot regime (Izacard et al. 2022).

Later controllers decide when to search again. IRCoT interleaves retrieval with chain-of-thought steps (Trivedi et al. 2022). FLARE retrieves when the next sentence looks uncertain (Jiang et al. 2023). DRAGIN triggers retrieval from

Method	What it decides	What it prices	What we add
Adaptive-RAG (Jeong et al. 2024)	No retrieval, one retrieval, or multi-step retrieval, from the question alone	Step count and relative time	A second look after the passages arrive, with dollars and a per-action charge
Self-RAG (Asai et al. 2024)	Whether to retrieve, and whether a segment is supported, as tokens in the generator	Hand-set critique weights	A frozen generator. Critique is one typed action with a measured cost
CRAG (Yan et al. 2024)	Keep, discard, or mix the retrieved set. Discard triggers a web search	An evaluator threshold	Rewrite, rerank, and verify on a closed corpus
Search-R1 (Jin et al. 2025)	When the language model itself emits a search during reasoning	Answer correctness	A small controller, so a change in exact match is a change in control
GRASP (Gandhi et al. 2026)	Semantic search, keyword search, or a paragraph read	Answer quality, grounded reading, using both searches, and a turn count	Rewrite, rerank, and verify, priced in dollars

Table 1: Published controllers this paper is positioned against. Self-RAG, CRAG, Search-R1, and GRASP are not reimplemented on this stack. Adaptive-RAG is. A classifier reads the question and chooses one-shot retrieval or the rewrite-first recipe before any passage is seen. That measurement is Table 4.

the model's own information need (Su et al. 2024). Self-Ask splits a multi-hop question into follow-ups (Press et al. 2022). Toolformer inserts API calls, including a search engine, into the generation (Schick et al. 2023). On the query side, Ma et al. rewrite, then retrieve, then read (Ma et al. 2023). HyDE embeds a hypothetical answering passage instead of the question (Gao et al. 2022b). monoBERT reranks BM25 hits with a cross-encoder (Nogueira and Cho 2019). RankGPT asks a language model to permute the candidates (Sun et al. 2023).

The training line splits the same way. PPO is the usual clipped policy gradient (Schulman et al. 2017). GRPO, introduced for math reasoning, replaces the learned critic with a group of rollouts of the same prompt (Shao et al. 2024). R1-Searcher and ReSearch use that style of update to teach a language model to search (Song et al. 2025; Chen et al. 2025). RARR looks up evidence and revises a statement that the evidence does not support (Gao et al. 2022a). TRUE compares the metrics people use for that kind of check (Honovich et al. 2022). We do not reimplement these systems. They are the decisions our five actions stand in for.

3 Method

Figure 1 is the protocol. An episode is one question. The controller picks one action at each step. The episode ends at stop, or when the step or budget cap forces a stop.

Actions, Retriever, and Generator

The action set is retrieve, rewrite, rerank, verify, and stop. Retrieve runs BM25 (Robertson and Zaragoza 2009) and appends the top five passages. Rewrite asks the generator for a new query. Rerank reorders the current pool by lexical overlap with the query. Verify runs a lexical natural-language-inference check of the current draft and returns support, neutral, or contradiction. Stop emits a short answer, or abstains. Yes/no questions must be answered yes or no. Caps are eight steps, three retrieves, two rewrites, two reranks, and two verifies. Illegal actions are masked: with no evidence the policy cannot rerank, verify, or stop.

The generator is Qwen2.5-3B-Instruct at temperature 0 (Qwen Team 2024). Decoding is greedy. It reads the passages. The controller does not. The verifier is lexical, not a cross-encoder and not the generator judging itself. Training ran on one NVIDIA T4. The policy itself is a CPU network.

The price card meters local tokens at \$0.15 and \$0.60 per million input and output tokens, and BM25 at 5×10^{-5} dollars per call. Latency in the reward and in the budget check is a fixed table, not the wall clock: 212.4 ms per retrieve, 999.2 ms per rewrite, 143.2 ms per rerank, 0 ms per verify, and 853.9 ms plus 52.0 ms per completion token for a generation. Those values are the median measured cost of the same calls, so the term keeps its size. A wall-clock charge would make two episodes with the same actions differ by a few milliseconds, and that difference enters the return. The table removes it. The logged latency is still the measured time, which is why the three exams below can match on every prediction and still disagree on milliseconds. The latency weight is 0.5 and the unit is 10^{-3} dollars per second. Absolute dollars are an instrumentation scale for a local model. Ratios are what the policy can learn.

Observation, Training, and Selection

The observation is 16 numbers. Four of them are retrieval scores: the mean of the top five, the top score, the gap between the top two, and the minimum of the top five. Each is a z-score against the training slice, clipped to $[-3, 3]$. The rest are the evidence count, remaining steps, remaining budget, verifier support and contradiction, the four tool counts, and a verify one-hot for support, contradiction, and neutral. There is no dataset indicator and no passage text. Before any retrieval the score channels are 0, the budget is full, and the vector does not contain the question. Every question therefore presents the same observation at step zero, and the first action cannot depend on which question is being asked. Adaptivity begins at the second step, after a retrieval has been observed.

The policy is a multilayer perceptron with one hidden layer of width 16, which is 357 parameters. Width above 32 is rejected. We train with REINFORCE (Williams 1992) for 20

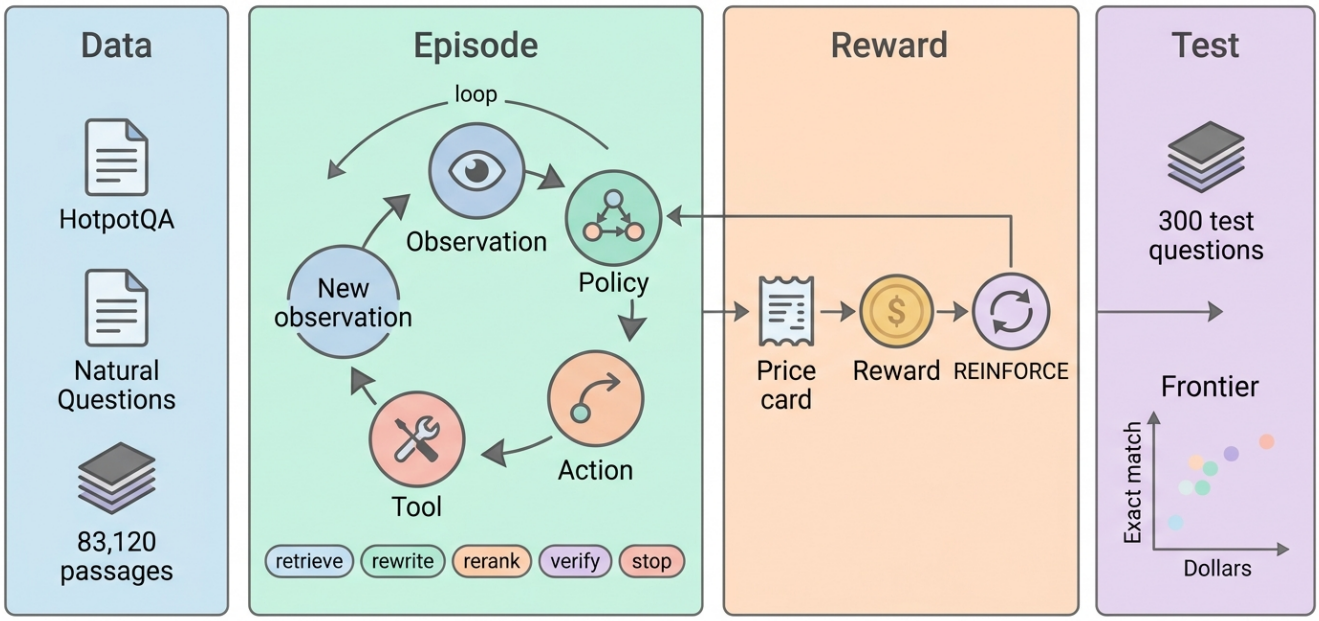


Figure 1: The episode. HotpotQA and Natural Questions share an index of 83,120 passages. The controller loops from an observation through one of five actions and back. The price card feeds the reward, REINFORCE carries that reward to the policy, and the 300 test questions feed the frontier.

epochs. The return is the terminal reward below. The advantage on a training question is that question's sampled reward minus the reward of one-shot retrieval on the same question. The entropy bonus coefficient is 0.01, the optimizer is Adam, and the learning rate is 0.003. Training uses 500 questions, 300 from HotpotQA and 200 from Natural Questions. After each epoch we roll out the mode on a held-out 180, 110 HotpotQA and 70 Natural Questions. The checkpoint we report is the first epoch that attains the highest greedy reward on that 180. It is chosen before the test set is opened. The test action is the mode of the masked softmax.

The rewrite-first exam below is three trainings at λ=20 and ρ=0, seeds 42, 43, and 44. We report the three exams, not a mean and a standard deviation. A second controller forces the first action to be retrieve and is scored in Table 5. The generator seed is temperature 0. The policy seed sets the initial weights and the per-question action sample during training.

Reward

The terminal reward is

$$\begin{aligned} R = {} & \alpha Q_{\text{ans}} + \beta Q_{\text{ground}} + \gamma Q_{\text{cal}} \\ & - \lambda(C_{\text{tok}} + C_{\text{ret}}) - C_{\text{lat}} \\ & - P_{\text{hall}} - P_{\text{act}} - P_{\text{bud}} + B. \end{aligned} \tag{1}$$

Q_{ans} is the average of exact match and token F1, the two scores defined for SQuAD (Rajpurkar et al. 2016). Q_{ground} mixes lexical overlap between the answer and the passages with, on HotpotQA, recall of the gold supporting titles supplied with that dataset (Yang et al. 2018). C_{tok} and C_{ret} are price-card dollars. C_{lat} is the table latency times the latency weight. P_{hall} penalizes unsupported and contradicted claims. P_{bud} is a unit penalty if a hard budget breaks. The unused-budget bonus B is our term. GRASP pays its turn-efficiency bonus only when answer F1 exceeds 0.5, so a short wrong answer does not collect it (Gandhi et al. 2026). We use that gate, $Q_{\text{ans}} \geq 0.5$, and we pay the bonus on remaining budget rather than on unused turns. The action charge is $P_{\text{act}} = \rho \max(0, n-1)$ for n actions in the episode. Calibration, the hallucination penalty, the action charge, and the dollar weights are ours.

Default weights are α=1, β=0.4, γ=0.15, λ=2, latency weight 0.5, hallucination weight 0.5, and ρ=0.02. The three-seed exam sets ρ=0 and λ=20. A correct answer scores $+0.3$ on the calibration term. Abstaining scores $+0.6$ when the passage list is empty, the mean retrieval score is below 3, or verify returns contradiction, and -0.2 otherwise. On this corpus the mean-score arm does not fire: raw BM25 means stay above 20, and means after the lexical reranker stay above 7. Empty evidence and a contradiction label still earn $+0.6$. A wrong answer scores -0.4 with evidence and -0.2 without it.

Data

The test set is locked at 300 questions: 150 HotpotQA distractor items (Yang et al. 2018) and 150 Natural Questions items (Kwiatkowski et al. 2019), disjoint from training and validation. The index holds HotpotQA contexts, unused HotpotQA distractors, and the DPR 100-word Wikipedia passages released with Natural Questions (Karpukhin et al. 2020; Gao et al. 2022c), including the passages needed by the 500 train-

	One-shot	Rule	Max tools	Six-step
HotpotQA	60/150	56/150	**62/150**	60/150
Natural Questions	40/150	41/150	**43/150**	40/150
Correct	100/300	97/300	**105/300**	100/300
$\$ \times 10^{-4}$	**1.72**	5.04	7.47	4.62
Reward	**0.581**	0.534	0.503	0.498

Table 2: Frozen controllers on the 83,120-passage index. Dollars are shown times 10^{-4}. Bold is the best value in the column: most correct answers, highest reward, or lowest dollars. The six-step recipe is retrieve, rewrite, rewrite, verify, verify, stop. It emits the same 300 answers as one-shot retrieval.

	Q_{ans}	Q_{ground}	Q_{cal}	$\$ \times 10^{-4}$	Reward
One-shot	0.366	0.646	−0.137	**1.72**	**0.581**
Rule	0.360	0.652	−0.146	5.04	0.534
Max tools	**0.385**	**0.658**	**−0.127**	7.47	0.503
Six-step	0.366	0.646	−0.137	4.62	0.498

Table 3: Reward terms for the frozen controllers, under the default weights λ=2 and ρ=0.02. Dollars are shown times 10^{-4}. Bold is the best value in the column. Q_{ans} is the average of exact match and token F1. Max tools leads on quality and trails on reward. The learned exam uses a different weight vector and is not in this table.

ing and 180 validation questions, 83,120 passages in total. Gold text is a Wikipedia passage. BM25 recall at five, measured on the 80,000-passage snapshot of this evaluation set, is 0.927 on HotpotQA and 0.587 on Natural Questions, so a second retrieval can matter. Three frozen controllers define the corners of the cost plot: one-shot retrieval, a rule that reranks and verifies when the mean score is at least 3, and a max-tool policy that spends three retrieves, one rewrite, one rerank, and one verify. A fourth frozen string, retrieve then rewrite then rewrite then verify then verify then stop, is the six-step recipe used in the ceilings.

4 Results

Extra Tools Buy Five Answers and Lose Reward

Table 2 is the frozen controllers on the 83,120-passage index. One-shot retrieval scores 100/300. Using every tool scores 105/300. From one-shot retrieval to max tools, Q_{ans} rises by 0.019, Q_{ground} by 0.012, and Q_{cal} by 0.010, about 0.025 once the weights in Equation 1 are applied. The action charge rises from 0.02 to 0.12, because the episode grows from 2 actions to 7 and ρ=0.02. The reward falls from 0.581 to 0.503. One extra retrieval costs roughly 3×10^{-4} in the λ=2 dollar term and 0.02 in ρ, so the flat charge is about sixty times the measured dollar price of that call. The rule policy pays for a rerank and a verify on every question, lands between them on cost, and scores 97/300. Table 3 is the same four controllers under the default weights, λ=2 and ρ=0.02. Max tools has the highest answer quality and the lowest reward. The six-step recipe matches one-shot retrieval on Q_{ans} and Q_{cal} and spends more, so its reward is the lowest of the four. The learned exam below uses λ=20 and ρ=0, so its return is not a row of this table.

The Ceiling on This Index Is 126

Figure 2 plots exact match against dollars. An oracle keeps the cheapest correct trajectory among one-shot retrieval, max tools, and the learned recipe, and the cheapest trajectory when none of them is correct. The six-step recipe emits the same 300 answers as one-shot retrieval, so it adds no question. One-shot retrieval and the learned recipe cover 122 questions between them. Max tools adds 4 questions that both of those miss. The oracle scores 126/300 at 1.81×10^{-4} dollars. Single policies span 97 to 105 correct, and the learned recipe scores 100. The room above one-shot retrieval, among trajectories we logged, is 26 answers. Assigning one-shot retrieval to every HotpotQA question and the six-step recipe to every Natural Questions question scores 100/300 at 3.29×10^{-4} dollars, because that recipe is more expensive and does not change an answer.

Three Seeds Select One Recipe

Seeds 42, 43, and 44 at λ=20 select epochs 17, 14, and 11 and each emit rewrite, rewrite, retrieve, stop on all 300 questions, scoring 100/300 at 2.02×10^{-4} dollars. The return under this preset, λ=20 and ρ=0, is 0.598 on each seed. That number is not comparable to the default-weight rewards in Table 3. Measured latency, which is logged and not priced, is 3061 ms, 3000 ms, and 2956 ms.

Against one-shot retrieval the recipe agrees on 179 of 300 predictions. Of the 121 disagreements, 22 become correct, 22 become incorrect, 1 stays correct under a different string, and 76 stay wrong. The repairs and the breaks are HotpotQA $+5$ and -13, Natural Questions $+17$ and -9. The union is 122/300. Both policies are right on 78 questions, so $78+22+22 = 122$. Figure 3 splits those 300 questions by dataset.

That union is the pairwise coverage of one-shot retrieval and the learned recipe. The ceiling in Figure 2 keeps the cheapest correct trajectory among one-shot retrieval, max tools, and this learned recipe. The six-step recipe adds no new correct question. Max tools adds 4 questions that one-shot retrieval and the learned recipe both miss, so the ceiling is 126/300 at 1.81×10^{-4} dollars. Any one policy still scores about 100. The questions move.

The recipe never branches on the retrieval score. We count a seed only when the second action is sometimes rewrite and sometimes retrieve, at least 15 times each, and one threshold on the mean score matches at least 250 of those questions. Here the first action is already rewrite, so the second action still sees a mean score of 0 and no passages. That action is rewrite on all 300 questions, on all three seeds. Retrieve is never the second action. The count is 0 of 3.

Verify is not what moves the 44 answers. The learned recipe never calls it. The six-step recipe calls it twice on every question and still emits the same 300 answers as one-shot retrieval.

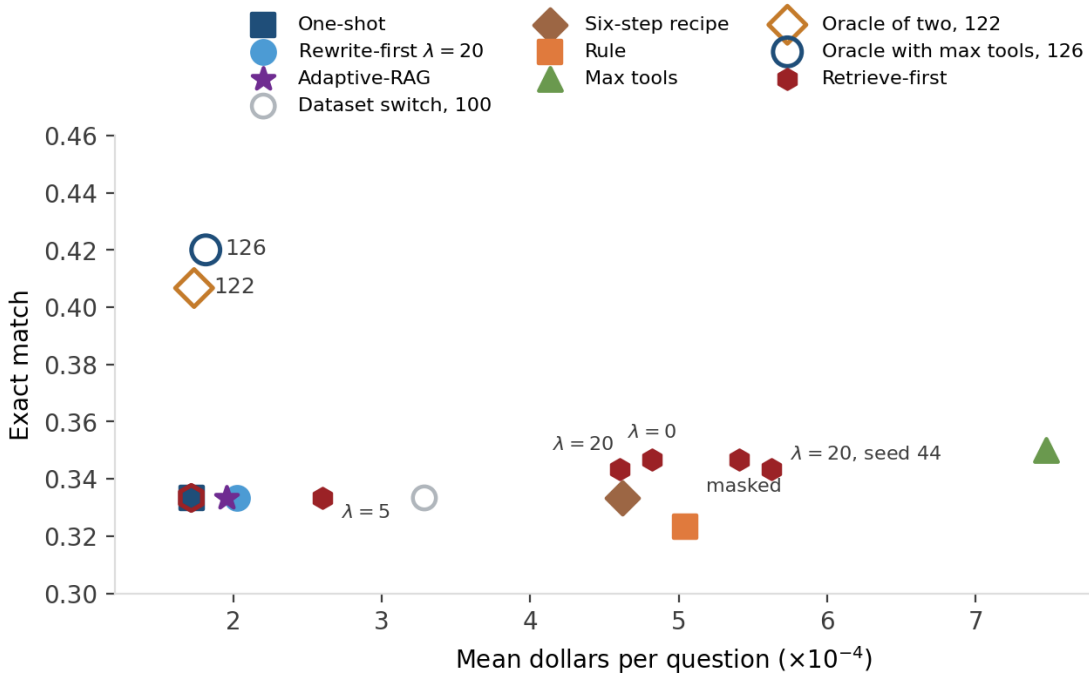


Figure 2: Exact match against mean dollars per question on the 83,120-passage index. Squares are one-shot retrieval and the rule. The triangle is max tools. The diamond is the six-step recipe. The filled circle is the rewrite-first λ=20 exam; seeds 42, 43, and 44 sit on the same point. Filled hexagons are the retrieve-first controller. The labeled ones are seed 42 at λ=0, 5, and 20, seed 44 at λ=20, and the masked seed 45. The open hexagon on the one-shot square is λ=80 and seed 43 at λ=20. The star is Adaptive-RAG. The gray circle is the dataset switch, which scores 100. The open diamond is the oracle of one-shot retrieval and the rewrite-first recipe, 122. The open circle adds max tools and scores 126. The cheap-label classifier sits on the one-shot point. The legend is outside the axes.

Question-Only Routing Scores 100

Adaptive-RAG trains a classifier on the question and then commits to one retrieval strategy (Jeong et al. 2024). The published classifier chooses among no retrieval, one retrieval, and a multi-step loop. Here the choice is the two recipes already logged: one-shot retrieval, and rewrite, rewrite, retrieve, stop. The classifier reads the question string. It does not see passages, BM25 scores, or the dataset name.

On the 180 validation questions, one-shot retrieval alone scores 58/180, 40/110 on HotpotQA and 18/70 on Natural Questions, at 1.70×10^{-4} dollars. The rewrite-first recipe alone scores 59/180, 36/110 and 23/70, at 2.02×10^{-4} dollars. The features are word and bigram TF-IDF. Logistic regression is fit at inverse regularization strengths 0.01, 0.1, 1, 10, and 100, and the kept classifier is the one with the most exact matches on those 180. Ties go to the lower mean dollar cost.

The first labeling gives a question to one-shot retrieval when that recipe is already correct, and to the rewrite-first recipe otherwise. A question both recipes miss is therefore labeled rewrite-first. On the 500 training questions that rule yields 188 one-shot labels and 312 rewrite-first labels. Strengths 10 and 100 both score 62/180, above either recipe alone. Strength 100 costs less on the 180, so it is the one we freeze. The 300 is opened only after that choice. It sends 63 questions to one-shot retrieval and 237 to the rewrite-first recipe, scores 55/150 and 45/150, and totals 100/300 at 1.96×10^{-4} dollars. Of the 22 repairs it keeps 17. Of the 22 breaks it avoids 5.

The second labeling is the cheapest strategy that works. A question both recipes miss is labeled one-shot, the cheaper recipe. The 500 then have 467 one-shot labels and 33 rewrite-first labels, one for each training repair. Every strength scores 58/180 and sends all 180 questions to one-shot retrieval, matching one-shot retrieval alone. On the 300 the frozen classifier does the same: 60/150 and 40/150, 100/300, at 1.72×10^{-4} dollars. It keeps 0 of the 22 repairs and avoids all 22 breaks.

Table 4 places both rows next to the two recipes and the two oracles. The oracle of the two recipes is 65/150 and 57/150, 122/300, at 1.74×10^{-4} dollars. Adding max tools makes 67/150 and 59/150, 126/300, at 1.81×10^{-4} dollars. The reward is not in this table. The rewrite-first trajectories were logged at λ=20 and ρ=0.

Diagnostics

An earlier observation passed the mean BM25 score through $\tanh(\text{score}/5)$. Top-1 scores on this corpus run from about 26 to 122, so that feature was 1.0 on all 300 questions and a policy could not branch on it. Standardizing the score makes

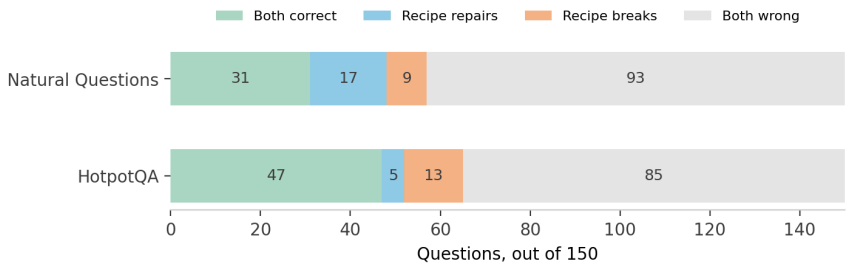


Figure 3: One-shot retrieval against the learned recipe. Each bar is 150 questions. HotpotQA is 47 both correct, 5 repaired, 13 broken, and 85 both wrong. Natural Questions is 31, 17, 9, and 93.

	Hotpot	NQ	Correct	$
One-shot	60/150	40/150	100/300	**1.72**
Rewrite-first	52/150	48/150	100/300	2.02
Adaptive-RAG	55/150	45/150	100/300	1.96
Cheap label	60/150	40/150	100/300	**1.72**
Oracle, two	65/150	57/150	122/300	1.74
Oracle + max	**67/150**	**59/150**	**126/300**	1.81

Table 4: Question-only routing on the 300. Dollars are the mean price-card cost, times 10^{-4}. Adaptive-RAG labels a double miss as rewrite-first. Cheap label marks that miss as one-shot. Oracle + max may also use the max-tools trajectory. Bold is the best value in the column. The two 1.72 cells tie.

a branch expressible. The three rewrite-first seeds do not use it. The first action is chosen before any score exists, and the selected string spends both rewrites before the first retrieve. The earlier saturated feature is the reason the branch had to be defined, not a second result.

Seeing the Score Still Misses the Gap

The rewrite-first recipe spends both rewrites before any passage exists. The next controller forces the first action to be retrieve, then chooses rewrite, retrieve, rerank, verify, or stop after the BM25 scores are in the observation. Nine features are appended to the 16 numbers: question length, the words who, what, when, and which, a yes/no bit, a capitalized-entity count, whether the draft abstained, and the word overlap of the draft with the top passage. The observation is 25 numbers. The hidden layer stays width 16, which is 501 parameters. There is still no dataset indicator. The split is the same 500, 180, and 300, and the checkpoint is still the first epoch with the highest greedy reward on the 180. The action charge stays off.

Table 5 is the retrieve-first controller. The first three rows are seeds 42, 43, and 44 at λ=20. The next three rows are seed 42 at λ=0, 5, and 80. The last row is masked seed 45. The first action is retrieve on 300/300 in every file. Correct counts at λ=20 are 103, 100, and 103. Seed 43 is retrieve, then stop, on 300/300. Its predictions and its dollar bill match one-shot retrieval. Seed 42 uses 15 action strings. The common one is retrieve, rewrite, rewrite, retrieve, retrieve, stop, on 164/300. Seed 44 uses one string on 300/300: retrieve, verify, verify, rewrite, rewrite, retrieve, retrieve, stop.

The answer is generated at stop, from the question and the passages then in hand. An idle action cannot change it: every verify, a retrieve of a query already searched, and a rewrite with no later retrieve. The Idle column is the mean price-card cost of those actions. Seed 42 spends 2.34 of its 5.51 tool calls this way, 31.3% of its bill. Seed 44 spends 3.03 of 7.00 tool calls, 34.3% of its bill. Seed 43 spends none.

Repairs are questions one-shot retrieval got wrong and this row got right. Breaks are the reverse. The paired sign test is the exact McNemar test: an exact two-sided binomial test on those discordant questions, success probability 1/2. The 95% interval is a percentile bootstrap of exact match: 10000 draws with replacement from these questions, seed 42, percentiles 2.5 and 97.5. One-shot retrieval on this file scores 100/300 at 1.72×10^{-4} dollars, with measured latency 1196 ms and interval 0.280–0.387. None of the three tests separates the row from one-shot retrieval. The returns use a different weight vector from the 0.581 in Table 3.

The branch rule is unchanged. A seed counts only when the second action is rewrite at least 15 times and retrieve at least 15 times, and one threshold on the mean score matches at least 250 of those questions. Seed 42's second action is rewrite 230, retrieve 66, and verify 4. The best threshold matches 236 of 296, which is under 250. Seed 43's second action is stop on 300/300. Seed 44's second action is verify on 300/300. The count is 0 of 3.

The λ=0, 5, and 80 rows are seed 42 only, with the action charge held at 0. λ=5 and λ=80 match one-shot retrieval on all 300 predictions. λ=80 also matches the dollar bill, at a measured latency of 1288 ms beside 1196 ms. λ=0 scores 104/300, HotpotQA 60/150 and Natural Questions 44/150, repairs 10 and breaks 6, HotpotQA +1 and −1, Natural Questions +9 and −5, p=0.454. Four answers is the largest move in the table, tied with seed 45. The sign test does not separate it from one-shot retrieval, so it is not a captured piece of the 26-answer gap.

λ=0 is the one sweep seed whose second action is both rewrite and retrieve often enough to meet the count: rewrite 282, retrieve 18, and a threshold of 109.9 matches 282 of 300. That threshold sits above every mean score in the file, and the match is the rewrite majority. The 18 retrieves have

λ	Seed	Ep.	Correct	\$	Idle	ms	Rep.	Br.	p	95%
20	42	17	103/300	4.60	1.44	4533	7	4	0.549	0.290–0.397
20	43	2	100/300	1.72	0.00	1226	0	0	1.000	0.280–0.387
20	44	19	103/300	5.62	1.93	5285	9	6	0.607	0.290–0.400
0	42	18	104/300	4.82	1.56	4383	10	6	0.454	0.293–0.403
5	42	7	100/300	2.60	0.88	1655	0	0	1.000	0.280–0.387
80	42	4	100/300	1.72	0.00	1288	0	0	1.000	0.280–0.387
20	45	12	104/300	5.41	1.42	4110	11	7	0.481	0.293–0.400

Table 5: Retrieve-first controller on the 83,120-passage index. The last row is masked seed 45. The λ=20 block is three seeds, and seed 42 appears once. Dollars and Idle are times 10^{-4}. Idle cannot change the answer written at stop. Repairs, breaks, p, and the 95% interval are against one-shot retrieval, 100/300 at 1.72 and 1196 ms. We report each row, not a mean.

mean scores 20.4–34.5, inside the rewrite range 22.4–108.9. The three-seed count at λ=20 stays 0 of 3. This sweep is one seed per weight.

At λ=5 the 300 answers match one-shot retrieval, and the idle dollars are the whole gap from 1.72 to 2.60. Removing them leaves the one-shot bill on every question. At λ=20, seed 44's extra price-card dollars times λ are 0.0078 in the reward, while a fully correct answer adds 1 through Q_{ans}. Up to λ=20 that dollar term is too small to steer the policy, and at λ=80 the controller falls back to one-shot retrieval. No weight in this sweep spends a tool only when the tool can change the answer.

Seed 45 is the same controller at λ=20 with a mask on three of those actions: no retrieve of a query already searched, no second verify, and no rewrite in the last slot before stop. It selects epoch 12 and scores 104/300. The second action is verify 266 and rewrite 34, retrieve 0, so the branch fails. The mask blocks the repeats, so the policy spends on idle actions the mask still allows: the one allowed verify on every question, and 38 rewrites are never followed by a retrieve. A rewrite is blocked only in the last slot, so the idle share drops from 34.3% to 26.2% and the answer count does not move. At λ=20 the policy is blind to cost, and removing one wasteful action only moves the spend onto another. Seed 45 changes both the seed and the mask, so it is not a clean ablation against seed 44.

5 Limitations

There are three limits on what these results support.

First, the controller never reads passage text. This is a study of the reward and of credit assignment in a small policy, not a fine-tune of the generator. The retrieve-first controller's second action sees a real BM25 score, and the score branch still appears in 0 of 3 seeds at λ=20.

Second, the rewrite-first exam is three seeds at one setting, λ=20 with the action charge off. The retrieve-first sweep is one seed at each of λ=0, 5, 20, and 80, again with the charge off. We report each row. We do not report a mean or a standard deviation. The frozen rows in Table 2 are one pass each, at temperature 0.

Third, the verifier and the reranker are lexical, and the price card is a scale for a local model. A neural verifier, a dense retriever, or a generator-sized policy could move the 26-answer gap and the dollar weight at which extra tools stop paying. That comparison was not run. Self-RAG, CRAG, Search-R1, and GRASP are not reimplemented. Adaptive-RAG in Table 4 is the two-recipe classifier.

6 Conclusion

On one index, the flat action charge decides tool use. Max tools scores 105/300 against 100/300 for one-shot retrieval, and the reward falls from 0.581 to 0.503 because the charge rises by more than the quality terms.

The gap among the trajectories we logged is 26 answers, 126/300 against 100/300. A fixed recipe and question-only routing stay at 100/300.

Forcing the first action to be retrieve, so the second action sees a score, does not take that gap. Three seeds at λ=20 score 103, 100, and 103, and the score branch appears in 0 of 3. One seed at λ=0, 5, 20, and 80 scores 104, 100, 103, and 100. Masked seed 45 also scores 104/300, and its idle share falls from 34.3% to 26.2% without taking the gap. The paper is a diagnosis of the charge and of the 26-answer gap. A neural verifier, a policy that reads the passage text, and GRPO were not run (Shao et al. 2024; Gandhi et al. 2026).

AI Use

A language model was used to revise sentences and to check bibliography entries against paper abstracts. An image model drafted Figure 1. The plots were drawn from the logged numbers. The tables are LaTeX, and every cell was checked against those numbers. No model output is reported as a measurement.

Ethical Statement

Pricing a tool can refuse a search that would have changed the answer, and the user is not told. Fixed recipes score 97 to 105 out of 300, and an oracle over one-shot retrieval, max tools, and the learned recipe reaches 126 near the one-shot price. The learned recipe and one-shot retrieval each score 100 and disagree on 44 questions. The abstention bonus does not fire on this corpus. The rewrite-first checkpoints never call the verifier. Retrieve-first seed 44 calls it twice on every question, and masked seed 45 calls it once. The dollars are a scale for a local 3B model, not a bill a person pays. The questions and passages are public benchmarks with no human subjects, and the checkpoints are not a serving policy.

References

Asai, A.; Wu, Z.; Wang, Y.; Sil, A.; and Hajishirzi, H. 2024. Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection. In *International Conference on Learning Representations*.

Borgeaud, S.; Mensch, A.; Hoffmann, J.; Cai, T.; Rutherford, E.; Millican, K.; van den Driessche, G.; Lespiau, J.-B.; Damoc, B.; Clark, A.; de Las Casas, D.; Guy, A.; Menick, J.; Ring, R.; Hennigan, T.; Huang, S.; Maggiore, L.; Jones, C.; Cassirer, A.; Brock, A.; Paganini, M.; Irving, G.; Vinyals, O.; Osindero, S.; Simonyan, K.; Rae, J. W.; Elsen, E.; and Sifre, L. 2021. Improving Language Models by Retrieving from Trillions of Tokens. arXiv:2112.04426.

Chen, L.; Zaharia, M.; and Zou, J. 2023. FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance. arXiv:2305.05176.

Chen, M.; Sun, L.; Li, T.; Sun, H.; Zhou, Y.; Zhu, C.; Wang, H.; Pan, J. Z.; Zhang, W.; Chen, H.; Yang, F.; Zhou, Z.; and Chen, W. 2025. ReSearch: Learning to Reason with Search for LLMs via Reinforcement Learning. arXiv:2503.19470.

Gandhi, V.; Lee, J.; Todmal, S.; Dernoncourt, F.; Rossi, R.; Wang, Z.; and Lan, A. 2026. GRASP: GRanularity-Aware Search Policy for Agentic RAG. arXiv:2607.10463.

Gao, L.; Dai, Z.; Pasupat, P.; Chen, A.; Chaganty, A. T.; Fan, Y.; Zhao, V. Y.; Lao, N.; Lee, H.; Juan, D.-C.; and Guu, K. 2022a. RARR: Researching and Revising What Language Models Say, Using Language Models. arXiv:2210.08726.

Gao, L.; Ma, X.; Lin, J.; and Callan, J. 2022b. Precise Zero-Shot Dense Retrieval without Relevance Labels. arXiv:2212.10496.

Gao, L.; Ma, X.; Lin, J.; and Callan, J. 2022c. Tevatron: An Efficient and Flexible Toolkit for Dense Retrieval. arXiv:2203.05765.

Guu, K.; Lee, K.; Tung, Z.; Pasupat, P.; and Chang, M.-W. 2020. REALM: Retrieval-Augmented Language Model Pre-Training. In *Proceedings of the 37th International Conference on Machine Learning*, 3929–3938.

Honovich, O.; Aharoni, R.; Herzig, J.; Taitelbaum, H.; Kukliansy, D.; Cohen, V.; Scialom, T.; Szpektor, I.; Hassidim, A.; and Matias, Y. 2022. TRUE: Re-evaluating Factual Consistency Evaluation. arXiv:2204.04991.

Izacard, G.; and Grave, E. 2020. Leveraging Passage Retrieval with Generative Models for Open Domain Question Answering. arXiv:2007.01282.

Izacard, G.; Lewis, P.; Lomeli, M.; Hosseini, L.; Petroni, F.; Schick, T.; Dwivedi-Yu, J.; Joulin, A.; Riedel, S.; and Grave, E. 2022. Atlas: Few-shot Learning with Retrieval Augmented Language Models. arXiv:2208.03299.

Jeong, S.; Baek, J.; Cho, S.; Hwang, S. J.; and Park, J. C. 2024. Adaptive-RAG: Learning to Adapt Retrieval-Augmented Large Language Models through Question Complexity. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, 7036–7050.

Jiang, Z.; Xu, F. F.; Gao, L.; Sun, Z.; Liu, Q.; Dwivedi-Yu, J.; Yang, Y.; Callan, J.; and Neubig, G. 2023. Active Retrieval Augmented Generation. arXiv:2305.06983.

Jin, B.; Zeng, H.; Yue, Z.; Yoon, J.; Arik, S.; Wang, D.; Zamani, H.; and Han, J. 2025. Search-R1: Training LLMs to Reason and Leverage Search Engines with Reinforcement Learning. arXiv:2503.09516.

Karpukhin, V.; Oğuz, B.; Min, S.; Lewis, P.; Wu, L.; Edunov, S.; Chen, D.; and Yih, W.-t. 2020. Dense Passage Retrieval for Open-Domain Question Answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, 6769–6781.

Kwiatkowski, T.; Palomaki, J.; Redfield, O.; Collins, M.; Parikh, A.; Alberti, C.; Epstein, D.; Polosukhin, I.; Devlin, J.; Lee, K.; Toutanova, K.; Jones, L.; Kelcey, M.; Chang, M.-W.; Dai, A. M.; Uszkoreit, J.; Le, Q.; and Petrov, S. 2019. Natural Questions: A Benchmark for Question Answering Research. *Transactions of the Association for Computational Linguistics*, 7: 452–466.

Lewis, P.; Perez, E.; Piktus, A.; Petroni, F.; Karpukhin, V.; Goyal, N.; Küttler, H.; Lewis, M.; Yih, W.-t.; Rocktäschel, T.; Riedel, S.; and Kiela, D. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems*, volume 33, 9459–9474.

Ma, X.; Gong, Y.; He, P.; Zhao, H.; and Duan, N. 2023. Query Rewriting for Retrieval-Augmented Large Language Models. arXiv:2305.14283.

Nogueira, R.; and Cho, K. 2019. Passage Re-ranking with BERT. arXiv:1901.04085.

Ong, I.; Almahairi, A.; Wu, V.; Chiang, W.-L.; Wu, T.; Gonzalez, J. E.; Kadous, M. W.; and Stoica, I. 2024. RouteLLM: Learning to Route LLMs with Preference Data. arXiv:2406.18665.

Press, O.; Zhang, M.; Min, S.; Schmidt, L.; Smith, N. A.; and Lewis, M. 2022. Measuring and Narrowing the Compositionality Gap in Language Models. arXiv:2210.03350.

Qwen Team. 2024. Qwen2.5 Technical Report. arXiv:2412.15115.

Rajpurkar, P.; Zhang, J.; Lopyrev, K.; and Liang, P. 2016. SQuAD: 100,000+ Questions for Machine Comprehension of Text. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*.

Robertson, S.; and Zaragoza, H. 2009. The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in Information Retrieval*, 3(4): 333–389.

Schick, T.; Dwivedi-Yu, J.; Dessì, R.; Raileanu, R.; Lomeli, M.; Zettlemoyer, L.; Cancedda, N.; and Scialom, T. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. arXiv:2302.04761.

Schulman, J.; Wolski, F.; Dhariwal, P.; Radford, A.; and Klimov, O. 2017. Proximal Policy Optimization Algorithms. arXiv:1707.06347.

Shao, Z.; Wang, P.; Zhu, Q.; Xu, R.; Song, J.; Bi, X.; Zhang, H.; Zhang, M.; Li, Y. K.; Wu, Y.; and Guo, D. 2024.

DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. arXiv:2402.03300.

Song, H.; Jiang, J.; Min, Y.; Chen, J.; Chen, Z.; Zhao, W. X.; Fang, L.; and Wen, J.-R. 2025. R1-Searcher: Incentivizing the Search Capability in LLMs via Reinforcement Learning. arXiv:2503.05592.

Su, W.; Tang, Y.; Ai, Q.; Wu, Z.; and Liu, Y. 2024. DRAGIN: Dynamic Retrieval Augmented Generation based on the Information Needs of Large Language Models. arXiv:2403.10081.

Sun, W.; Yan, L.; Ma, X.; Wang, S.; Ren, P.; Chen, Z.; Yin, D.; and Ren, Z. 2023. Is ChatGPT Good at Search? Investigating Large Language Models as Re-Ranking Agents. arXiv:2304.09542.

Trivedi, H.; Balasubramanian, N.; Khot, T.; and Sabharwal, A. 2022. Interleaving Retrieval with Chain-of-Thought Reasoning for Knowledge-Intensive Multi-Step Questions. arXiv:2212.10509.

Williams, R. J. 1992. Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning. *Machine Learning*, 8(3–4): 229–256.

Yan, S.-Q.; Gu, J.-C.; Zhu, Y.; and Ling, Z.-H. 2024. Corrective Retrieval Augmented Generation. arXiv:2401.15884.

Yang, Z.; Qi, P.; Zhang, S.; Bengio, Y.; Cohen, W. W.; Salakhutdinov, R.; and Manning, C. D. 2018. HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, 2369–2380.

Yao, S.; Zhao, J.; Yu, D.; Du, N.; Shafran, I.; Narasimhan, K.; and Cao, Y. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations*.